

2.3.1 Number of Steps (Big O Notation)

بگ او (Big O) بتاتا ہے کہ ان پٹ n بڑھنے پر وقت یا میموری کیسے بڑھتی ہے۔

CS XII

Chapter 2: Computational Thinking & Algorithms

Lecture 09 of 24

★ Golden

2.3.1 Number of Steps (Big O Notation) ★ Golden

LEARN IT

When we discuss efficiency we ask how performance changes as the **input size n** increases. **Big O notation** is a mathematical method used to describe how the running time (or memory) **grows** as n grows. We ignore small constants and keep the most important term: if an algorithm does $3n + 5$ steps we write $O(n)$.

In computer science, **log n** almost always means log base 2 (how many times you can halve n until you reach 1).

Five common classes

Class	Name	What happens when n grows	Typical example	Grade
$O(1)$	Constant	Time stays the same for 10 items or 10 million.	Access <code>arr[0]</code> (or any known index).	Excellent
$O(\log n)$	Logarithmic	Time grows very slowly; the problem is halved each step.	Binary search on a sorted list.	Great
$O(n)$	Linear	If n doubles, time roughly doubles.	Linear search of an unsorted list.	Fair / Good
$O(n \log n)$	Linearithmic	A little worse than linear; much better than n^2 . Common in good sorting.	Merge Sort (split in half, then merge).	Good
$O(n^2)$	Quadratic	Time grows with the square of n . Nested loops. Painful for large n .	Bubble Sort; compare every item with every other item.	Poor

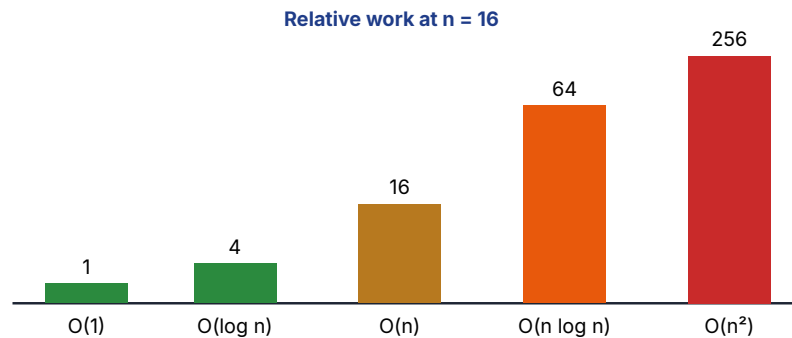
The source stresses $O(1)$, $O(\log n)$, $O(n)$ and $O(n^2)$. $O(n \log n)$ is the class of efficient general-purpose sorts, so you should recognise it when a question says "better than Bubble Sort but more than linear".

Growth table (log is $\log_2$)

Each cell is the approximate number of operations for that n . Read down a column to see how fast that class explodes.

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
2	1	1	2	2	4
4	1	2	4	8	16
8	1	3	8	24	64
16	1	4	16	64	256
32	1	5	32	160	1,024
1,024	1	10	1,024	10,240	1,048,576
1,000,000	1	~20	1,000,000	~20,000,000	1,000,000,000,000

From $n = 8$ to $n = 1,000,000$, $O(1)$ does not move, $O(\log n)$ only goes from 3 to about 20, $O(n)$ becomes a million, and $O(n^2)$ becomes a million million — which is why nested loops “fail for large datasets”.



Same $n = 16$. Quadratic work is already four times $n \log n$ and sixteen times linear.

WORKED EXAMPLE

Worked example A — $O(1)$: first element of an array. Whether the array holds 10 or 10 million values, `arr[0]` is one jump to a known address.

```
arr = [23, 76, 12, 90, 34, 68]
print(arr[0])
```

OUTPUT
23

Worked example B — $O(n)$: linear search. Each extra item adds one extra check.

```
def linear_search(items, key):
    for i, value in enumerate(items):
        if value == key:
            return i
    return -1

print(linear_search([10, 50, 20, 70, 90, 30], 20))
print(linear_search([10, 50, 20, 70, 90, 30], 99))
```

OUTPUT
2
-1

Worked example C — $O(\log n)$: binary search on a sorted list. Each step throws away half of the remaining items. For 8 sorted numbers the worst case is about 4 comparisons ($\text{floor}(\log_2 8) + 1$).

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    steps = 0
    while low <= high:
        mid = (low + high) // 2
        steps = steps + 1
        if arr[mid] == key:
            return mid, steps
        if arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1, steps

data = [3, 8, 12, 17, 23, 31, 40, 52]
print(binary_search(data, 40))
print(binary_search(data, 3))
```

OUTPUT

```
(6, 3)
(0, 3)
```

Key 40 is found at index 6 after 3 comparisons. Key 3 is found at index 0 after 3 comparisons. Never more than a handful of steps, even if the list later grows to a million (about 20 steps).

Worked example D — $O(n \log n)$: Merge Sort idea. To sort 8 items, Merge Sort makes $\log_2 8 = 3$ split-levels, and each level walks all 8 items once, so work $\approx 8 \times 3 = 24$, which matches the growth table. Bubble Sort on 8 items is on the order of $8^2 = 64$ comparisons. That gap becomes enormous at $n = 1,000,000$.

Worked example E — $O(n^2)$: nested loops.

```
n = 4
count = 0
for i in range(n):
    for j in range(n):
        count = count + 1
print(count)
```

OUTPUT

```
16
```

Inner body runs $n \times n = 16$ times. That is the pattern of Bubble Sort and of “compare every item with every other item”.

REMEMBER IT

Order from fastest to slowest (for large n): $O(1) \rightarrow O(\log n) \rightarrow O(n) \rightarrow O(n \log n) \rightarrow O(n^2)$.

Mnemonic: **“Constant Logs Never Need Squares”** — Constant, Logarithmic, N, N-log-N, Squares.

COMMON MISTAKE

Mistake 1: writing $O(n)$ for binary search because “there is a loop”. The loop runs about $\log n$ times, not n times.

Mistake 2: writing $O(2n)$ or $O(n+5)$. Big O drops constants: both are $O(n)$.

Mistake 3: using binary search on an **unsorted** list. Binary search is $O(\log n)$ only on sorted data.

اردو میں سمجھیں

بگ او (Big O) بتاتا ہے کہ ان پٹ n بڑھنے پر وقت یا میموری کیسے بڑھتی ہے۔ $O(1)$ مستقل (ایرے کا $arr[0]$)، $O(\log n)$ ہر قدم پر آدھا (بائنری سرچ)، $O(n)$ لکیری (لینیئر سرچ)، $O(n \log n)$ اچھے سارٹ جیسے مرچ سارٹ، اور $O(n^2)$ مربع (ببل سارٹ، اندرونی لوپ)۔ بڑے n پر $O(n^2)$ ناقابل استعمال ہو جاتا ہے۔

EXAM TIP

Memorise the five rows of the class table (name, growth, one example, grade). Typical MCQ: “Accessing $arr[0]$ is ...?” $\rightarrow O(1)$. “Binary search is ...?” $\rightarrow O(\log n)$. “Nested loops / Bubble Sort ...?” $\rightarrow O(n^2)$. Long questions ask you to fill or interpret a growth table, or to say what happens if n becomes 10 times larger (linear $\rightarrow 10\times$ time; quadratic $\rightarrow 100\times$ time). Always mention that $\log n$ is “halving” and n^2 is “nested loops”.

CHECK YOURSELF

1. Give one example each of $O(1)$, $O(\log n)$, $O(n)$ and $O(n^2)$.
2. From the growth table, how many operations does $O(n^2)$ need when $n = 32$?
3. Why is Merge Sort described as $O(n \log n)$?

Answers: 1. $O(1)$ arr[0]; $O(\log n)$ binary search; $O(n)$ linear search; $O(n^2)$ Bubble Sort / nested loops. 2. 1,024. 3. It makes about $\log n$ split-levels and does about n work on each level, so $n \times \log n$.

Next lecture: 2.3.2 Number of Conditions